Day 1 — Production Python for AI services

Outcome

Be able to design a Python AI service that is readable, testable, provider-neutral, validated at its boundaries, observable without leaking secrets, and safe to refactor.

1. Core Python mental model

Python is dynamically typed: a name refers to an object, and the object has a type. Type hints document expected contracts and support static analysis, but do not enforce runtime validity.

High-signal types and collections

Type	Best use	Important pitfall
`int`	Counts, IDs when arithmetic is meaningful, token totals	Do not treat digit-only identifiers as quantities.
`float`	Scores, probabilities, latency	Avoid exact equality assumptions.
`str`	Text, names, identifiers	Non-empty `"False"` is truthy.
`bool`	Explicit binary state	Avoid unclear truthiness for domain decisions.
`list`	Ordered mutable collection	Assignment shares the same list; copying must be intentional.
`tuple`	Fixed grouping or immutable key	Contained mutable objects can still change.
`dict`	Keyed lookup and structured metadata	A broad `dict[str, Any]` hides contracts.
`set`	Deduplication and membership	Iteration order is not a stable output contract.

AI examples:

- List: ranked retrieved chunks.
- Tuple: composite cache key.
- Dictionary: document metadata or provider configuration.
- Set: processed chunk IDs or allowed-role intersection.

Collection behavior worth recalling

- Assignment does not copy a mutable object: two names can refer to the same list or dictionary.
- `items.copy()` and `items[:]` create a shallow list copy. Nested mutable values are still shared unless a deeper copy is deliberately required.
- `list.sort()` changes a list in place and returns `None`; `sorted(iterable)` returns a new list. Do not assign the result of `.sort()` expecting the sorted data.
- Strings and tuples are immutable, but a tuple may still contain a mutable object.

- A dictionary comprehension is useful for a simple key/value transformation; duplicate generated keys overwrite earlier values.
- A set is ideal for membership or deduplication, but it discards counts and should not define user-visible ordering.

These details matter in request-scoped state, fixtures, caches, and agent/RAG payloads because an accidental alias or in-place update can leak changes across layers.

Slicing and comprehensions

Slicing is useful for bounded retrieval results:

```
top_chunks = ranked_chunks[:5]
```

Comprehensions are good for simple transformations:

```
chunk_ids = [chunk.id for chunk in chunks if chunk.score >= threshold]
```

Move filtering, conversion, and branching into named functions when the expression stops being immediately readable.

Iterators, generators, and context managers

An iterator produces values one at a time through the iteration protocol. A generator creates an iterator with `yield`; it evaluates lazily and can reduce memory while processing large document or event streams.

```
def iter_valid_chunks(chunks):
    for chunk in chunks:
        if chunk.text.strip():
            yield chunk
```

Laziness does not remove every cost: buffering, retained references, or downstream materialization can still consume memory.

A context manager makes resource ownership and cleanup explicit:

```
with database_connection() as connection:
    rows = connection.execute(query)
```

Use context managers for files, transactions, connections, process pools, and other resources that must be released after success or failure.

Function contracts

Prefer:

- explicit typed parameters for core services;
- keyword arguments when they clarify calls;
- one clear return type;
- pure functions for transformations;
- side effects isolated behind interfaces.

`*args` collects extra positional arguments into a tuple. `**kwargs` collects keyword arguments into a dictionary. They are useful for wrappers but weaken discoverability when used in a central service contract.

Mutable default trap

Defaults are created once when the function is defined:

```
# Unsafe: the list is shared by calls.
def add_chunk(chunk, chunks=[]):
    chunks.append(chunk)
    return chunks
```

Use:

```
def add_chunk(chunk, chunks=None):
    chunks = [] if chunks is None else chunks
    chunks.append(chunk)
    return chunks
```

For dataclasses:

```
from dataclasses import dataclass, field

@dataclass
class RetrievalResult:
    chunks: list[str] = field(default_factory=list)
```

2. Modules, packages, environments, and configuration

Module and package boundaries

- A module is normally one Python file.
- A package groups related modules under a namespace.
- Prefer absolute, consistent imports.
- Avoid wildcard imports.
- Avoid expensive work, API calls, model downloads, or connection creation at import time.
- Break circular imports by correcting ownership boundaries rather than moving imports randomly.

Suggested structure:

```
src/
├── domain/
├── interfaces/
├── services/
├── adapters/
├── api/
├── config/
└── workers/
tests/
├── unit/
├── integration/
└── end_to_end/
```

Environment tools

Tool	Responsibility
`venv`	Isolate packages for a project using an existing interpreter.
`pyenv`	Install and select Python interpreter versions.
`uv`	Integrated interpreter, environment, dependency, lockfile, and command workflow.

Keep local development and CI on declared Python and dependency versions. Do not rely on globally installed packages.

Configuration

Configuration should be external, explicit, validated at startup, and separated from runtime state.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ModelConfig:
    model_name: str
    timeout_seconds: float
    max_output_tokens: int
```

Practices:

- Use `.env` only as a local-development convenience.
- Commit an example file, not real secrets.
- Parse booleans and numbers explicitly.
- Fail startup when required configuration is missing.
- Use a secrets manager in deployed environments.
- Do not log configuration values that may contain credentials.

3. OOP that earns its complexity

Four principles

- **Encapsulation:** keep state and behavior together and preserve invariants.
- **Abstraction:** expose essential behavior while hiding implementation detail.
- **Inheritance:** specialize a genuine “is-a” relationship.
- **Polymorphism:** use different implementations through one stable contract.

Python production code often combines these with protocols and composition.

Python uses conventions rather than strict field privacy:

Form	Meaning
`self.model_name`	Public API.
`self._client`	Internal/protected by convention; callers can still access it.

`self.__api_key`	Name-mangled to reduce accidental access/ overriding; not a security control.

Encapsulation hides provider headers, token refresh, response parsing, retries, and latency recording behind a small operation such as `generate()`. Secrets still belong in a secrets manager, not in a supposedly private attribute.

Composition over deep inheritance

RAG components vary independently:

```
embedder
retriever
reranker
prompt builder
generator
```

Composition lets each be replaced or tested without a subclass for every combination.

Use inheritance when there is a stable substitutable relationship and shared semantics. Avoid deep hierarchies, giant base classes, and capability methods that many implementations cannot support.

ABC versus protocol

An abstract base class:

- requires explicit inheritance;
- may provide shared implementation;
- can enforce abstract methods at runtime.

A protocol:

- uses structural typing;
- allows any object with the required methods to satisfy the contract;
- reduces coupling and works well with dependency injection and fakes.

```
from typing import Protocol

class TextGenerator(Protocol):
    def generate(self, prompt: str) -> str:
        ...
```

Keep synchronous and asynchronous contracts separate:

```
class AsyncTextGenerator(Protocol):
    async def generate(self, prompt: str) -> str:
        ...
```

Class, static, and instance methods

- Instance method: uses instance state.
- `@classmethod`: receives the class; useful for alternative constructors or subclass-aware factories.
- `@staticmethod`: belongs conceptually to the class but needs neither instance nor class.

Do not create a utility class when a module-level function is clearer.

Instance variables belong to one object. Class variables are shared across instances; a mutable class-level dictionary or list can leak state between requests/tests and create races.

Method overriding lets a subclass provide behavior for a base contract. The override must preserve accepted inputs, response semantics, and predictable errors to remain substitutable.

Use `super()` when a subclass must initialize or extend parent behavior:

```
class LocalProvider(ModelProvider):
    def __init__(self, model_name: str, device: str) -> None:
        super().__init__(model_name)
        self.device = device
```

Calling the parent class by name is less flexible under refactoring or multiple inheritance.

Properties and dataclasses

Use a public attribute when no control is needed. Use `@property` when reading or writing requires validation or computation.

Dataclasses fit:

- immutable configuration;
- requests and responses;
- retrieved chunks;
- tool payloads;
- evaluation records.

Use `__post_init__` for small invariant checks. Use Pydantic when untrusted runtime input needs richer validation and error reporting.

Dunder methods

- `__repr__`: developer representation; never reveal secrets.
- `__str__`: user-friendly representation.
- `__eq__`: semantic equality.
- `__hash__`: only when equality and immutability semantics support hashing.
- `__call__`: object behaves like a callable.
- `__iter__`: object exposes iteration.
- `__enter__` / `__exit__`: deterministic resource cleanup.

Keep dunder behavior unsurprising.

4. SOLID and AI boundaries

Single Responsibility

Do not let one class load documents, chunk them, call an embedding service, write to a vector store, build prompts, and generate answers. Separate reasons to change.

Open/Closed

Add a provider or retrieval strategy through an implementation of a stable interface instead of changing a large conditional in every service.

Liskov Substitution

Every generator implementation must accept the documented request, return the documented response, preserve expected semantics, and raise predictable application errors.

Interface Segregation

Prefer small capability interfaces:

```
TextGenerator
StreamingTextGenerator
Embedder
ToolCaller
```

Do not force a provider to pretend it supports embeddings, streaming, image generation, batch jobs, and fine-tuning.

Dependency Inversion

Business services depend on internal interfaces. Vendor SDKs stay inside adapters.

```
domain/service → internal interface ← provider adapter
```

5. Provider-neutral RAG example

```
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class Evidence:
    chunk_id: str
    text: str
    source: str
    score: float

@dataclass(frozen=True)
class RagAnswer:
    answer: str
    citations: tuple[str, ...]

class Retriever(Protocol):
    def retrieve(self, question: str, tenant_id: str) -> list[Evidence]:
        ...

class Generator(Protocol):
    def generate(self, prompt: str) -> str:
        ...

class RagService:
    def __init__(
        self,
        retriever: Retriever,
        generator: Generator,
    ) -> None:
        self._retriever = retriever
        self._generator = generator

    def answer(self, question: str, tenant_id: str) -> RagAnswer:
        evidence = self._retriever.retrieve(question, tenant_id)
        prompt = build_grounded_prompt(question, evidence)
        text = self._generator.generate(prompt)
        citations = tuple(item.source for item in evidence)
        return RagAnswer(answer=text, citations=citations)
```

Why it works:

- domain objects do not expose provider SDK types;
- dependencies are injected;
- retrieval can enforce tenant scope;
- deterministic fakes can test orchestration;
- adapters can normalize provider errors.

6. Static typing and runtime validation

Type hints

Type hints improve:

- interface clarity;
- IDE support;
- safer refactoring;
- static checks;
- code review.

`Optional[str]` means `str | None`; it does not make an argument optional unless a default is supplied.

`TypedDict` describes dictionary shape to static tooling but does not validate at runtime.

High-signal type forms:

Form	Contract
`list[Evidence]`	Ordered collection of evidence.
`dict[str, str]`	String keys and values.
`str | None`	Value may be absent/`None`.
`str | bytes`	Either supported runtime type.
`TypedDict`	Statically described dictionary shape; no runtime validation.

Run a static checker such as `mypy` in development or CI:

```
mypy src/
```

For an older codebase, type new modules and public/API/database/provider/queue boundaries first, then enable stricter checks module by module. Prevent new untyped critical code while gradually reducing existing errors.

Pydantic

Use Pydantic at untrusted boundaries:

- HTTP requests and responses;
- tool arguments and results;
- configuration;
- queue messages;
- external provider payloads.

Decide whether coercion or strictness is appropriate. Reject or explicitly handle unexpected fields. Do not confuse validation with authorization: a syntactically valid tool request can still be forbidden.

Avoid overusing validators for complex business workflows. Keep schema checks at the boundary and domain/business rules in named services or domain functions. Do not expose raw internal validation details to external clients.

Example:

```
from pydantic import BaseModel, ConfigDict, Field

class ChatRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    question: str = Field(min_length=1, max_length=4_000)
    top_k: int = Field(default=5, ge=1, le=50)
```

Decide explicitly whether coercion is convenient or strict validation is required. Machine-to-machine and high-impact tool contracts often benefit from stricter types. Use field/cross-field validators for local schema invariants; keep database lookups, permissions, and workflow decisions outside the schema.

Boundary sequence:

```
untrusted input
→ schema validation
→ authentication/authorization
→ domain conversion
→ business rules
→ side effect
```

7. Error handling and logging

Exception strategy

Catch an exception only when the layer can:

- recover;
- add useful context;
- translate it to an application exception;
- clean up;
- map it to an API response.

Use narrow exceptions and preserve chaining:

```
class ModelProviderError(Exception):
    pass

class ModelTimeoutError(ModelProviderError):
    pass

try:
    response = provider.generate(prompt)
except ProviderTimeout as exc:
    raise ModelTimeoutError("generation timed out") from exc
```

Do not return `None` for every failure or catch `Exception` merely to hide it.

Keep only the risky operation in `try`. `else` runs only when that operation succeeds; `finally` runs on success or failure and is reserved for cleanup. Prefer a context manager when one exists:

```
try:
    payload = parse_provider_json(raw_response)
except InvalidJSONError as exc:
    raise ProviderResponseError("provider returned invalid JSON") from exc
else:
    validate_payload(payload)
finally:
    release_temporary_buffer()
```

Structured logging

Prefer parameterized logging and stable fields:

```
logger.info(
    "generation completed",
    extra={
        "request_id": request_id,
        "tenant_id": tenant_id,
        "model": model_name,
        "prompt_version": prompt_version,
        "duration_ms": duration_ms,
    },
)
```

Include request/trace IDs, component versions, duration, error category, retries, token counts, and relevant IDs.

A correlation ID connects logs for one logical request or workflow across API, queue, worker, model, retrieval, and tool boundaries.

Do not log:

- API keys or tokens;
- unrestricted prompts/responses;
- personal or confidential data;
- full tool arguments by default.

Avoid logging the same exception at every layer.

Logging levels carry operational meaning:

Level	**Use**
`DEBUG`	Detailed diagnostics such as filters, chunk counts, and retry attempts.
`INFO`	Normal business/system events such as completed requests or ingestion.
`WARNING`	Recovered degradation such as fallback use or a skipped record.
`ERROR`	An operation failed but the process can continue.
`CRITICAL`	The service cannot perform an essential function.

Use a module-level logger and `logger.exception()` inside the boundary that handles an exception when a traceback is needed. Do not emit the same failure as an error at every layer.

8. Testing strategy

Test layers

Layer	Purpose
Unit	Deterministic business logic, prompt construction, filters, ranking, parsing, validation.
Integration	Actual database, vector adapter, queue, or model-provider contract.
Contract	Request/response compatibility across boundaries.
End-to-end	Deployed user flow.
Evaluation	Probabilistic LLM/RAG quality against a reviewed dataset.

Structure individual tests with Arrange–Act–Assert:

```
Arrange: prepare input and fakes.
Act: call one behavior.
Assert: verify the result and important interaction.
```

Keep one test focused on one behavior so failures remain diagnostic.

A `pytest` fixture provides reusable setup and teardown. Prefer isolated function-scoped state by default; use broader scopes only when the sharing is intentional. A fixture that uses `yield` can release a database connection, temporary index, or other test resource even when the test fails.

Fakes before deep mocks

A fake generator is often clearer than mocking internal SDK calls:

```
class FakeGenerator:
    def __init__(self, response: str) -> None:
        self.response = response
        self.prompts: list[str] = []

    def generate(self, prompt: str) -> str:
        self.prompts.append(prompt)
        return self.response
```

Test:

- normal result;
- empty retrieval;
- invalid input;
- provider timeout;

- malformed structured output;
- authorization failure;
- retry boundaries;
- logging/redaction;
- version information.

Mocking pitfalls:

- testing the mock rather than behavior;
- mocking too deeply;
- unrealistic return values;
- calling a live LLM in every unit test;
- relying on test order or shared state.

9. Production best practices

- Keep constructors lightweight; inject initialized clients.
- Keep configuration immutable and runtime state explicit.
- Validate at boundaries and enforce authorization separately.
- Set timeouts for every external call.
- Retry only deliberate transient failures.
- Make retried operations idempotent.
- Hide third-party types and errors in adapters.
- Treat prompts as versioned application artifacts.
- Validate structured model output before use.
- Keep confidential data out of logs.
- Track latency, tokens, retries, provider errors, prompt/model versions, and retrieval statistics.
- Test deterministic logic without provider calls; retain a small real integration suite.
- Avoid speculative abstractions; introduce them at real variation points.

Project-grounded examples

Scenario 1: reusable Python boundaries in DPDK benchmark automation

Project scenario. In **DPDK Automation for Network Packet Processing**, the implementation had several different reasons to change: AMD Cinnabar BIOS handling, Dell/HP Redfish-based BIOS operations, Xena packet generation, DPDK command execution, and parsing for testpmd, crypto, and vhost output. The project used Python for the BIOS scripts, the Clif-based BIOS CLI, the reusable Xena integration module, stats processing, and custom parsers; Ansible roles handled repeatable host setup and benchmark workflows.

How the concepts apply. This is a concrete reason to use cohesive modules and composition. A packet-generator integration should expose a small benchmark-facing capability instead of leaking Xena-specific details throughout every workload. BIOS automation and result parsing are separate responsibilities: they have different inputs, failure modes, and change triggers. The documented “plug-and-play” Xena module and reusable roles are evidence of designing around real reuse points rather than creating abstraction only for style.

Decision and trade-offs. The decision was to generalize shared setup, command templates, statistics collection, and packet-generation behavior while keeping workload-specific parsers for formats that actually differed. That reduced duplication and supported extension across seven networking benchmarks, but a shared framework also had to preserve OS-, platform-, and workload-specific behavior. Too little abstraction would duplicate fragile automation; too much would hide the parameters that performance engineers needed to control.

Senior/Staff interview framing.

- **Senior:** explain one boundary in depth: "I made packet generation reusable, kept benchmark parsing workload-specific, and used configuration/templates for the real variation points." Describe inputs, errors, tests you would expect, and how that made a new benchmark easier to integrate.
- **Staff:** explain how you identified platform-wide capabilities—BIOS automation, environment setup, command templating, statistics, and reporting—and separated them from benchmark-specific logic. Connect the choice to team onboarding, consistency across Ubuntu/RHEL and compiler combinations, and reuse for future platforms.

Evidence boundary. The project narrative does not document use of Python protocols, ABCs, Pydantic, `mypy`, or a particular automated-test framework. Present those as ways you would formalize or strengthen the boundaries, not as technologies already used.

Scenario 2: separating AI reasoning from deterministic Python services

Project scenario. In **DPDK BenchOps Copilot**, the AI layer answered benchmark questions and synthesized tuning or regression explanations, while command construction, run lookup, log retrieval, plan validation, result parsing, and run comparison remained deterministic. LlamaIndex handled ingestion/retrieval, LangChain connected model and tool components, LangGraph controlled the workflow, and MCP exposed narrow operational tools.

How the concepts apply. The project is a practical SOLID example: retrieval, orchestration, model interaction, and tool execution have distinct contracts and safety responsibilities. The important boundary was not "one class per framework"; it was that business logic could ask for evidence or a safe operation without treating an LLM response as executable truth.

Decision and trade-offs. Keeping AI synthesis separate from execution sacrificed some free-form flexibility and introduced more interfaces, but it made high-risk behavior auditable and kept commands reproducible. The extra structure was justified by the cost of a hallucinated DPDK flag or an unsafe BIOS recommendation.

Senior/Staff interview framing.

- **Senior:** trace one request through retrieval, deterministic tool invocation, verification, and response construction, calling out validation and error translation at each boundary.
- **Staff:** lead with the governing principle—"probabilistic reasoning may propose; deterministic services validate and execute"—then explain how that principle shaped framework selection, operational ownership, release evaluation, and future extensibility.

10. Interview questions

1. Why is a tuple different from a list, and where would each appear in RAG?
2. Why can a mutable default leak state between requests?
3. When is a protocol better than an abstract base class?
4. Why is composition a better fit for independently varying RAG components?
5. What is the difference between type hints, `TypedDict`, dataclasses, and Pydantic?
6. How do you normalize errors from multiple LLM providers?
7. What belongs in structured logs for an AI request?
8. How do you test an LLM integration without slow and nondeterministic unit tests?
9. What makes a provider adapter substitutable?
10. What is a sign that an OOP design is overengineered?
11. How do assignment, a shallow copy, and an in-place sort differ?
12. When can a dictionary comprehension silently lose data?

11. Exit checklist

- [ ] Explain mutability, truthiness, collection choices, slicing, and mutable defaults.
- [ ] Explain `venv`, `pyenv`, and `uv`.
- [ ] Propose a production package layout.
- [ ] Explain ABC, protocol, dataclass, property, and composition.
- [ ] Apply all five SOLID principles to an AI service.
- [ ] Distinguish static typing from runtime validation.
- [ ] Design an exception hierarchy and safe structured logs.
- [ ] Build deterministic fakes and separate test layers.
- [ ] Keep SDK details out of domain and service code.
- [ ] Explain aliasing, shallow copies, in-place mutation, and stable output ordering.

Source notes

- Python Core & Environment
- OOP in Python for AI
- Python Advanced: Typing & Testing
- GenAI Design Patterns
- Capstone Revision Day 1
- DPDK Automation for Network Packet Processing
- DPDK BenchOps Copilot